

503 lines (316 loc) · 23 KB

Preview | Code | Blame  

Net_Practice

[My solution](#) and basic explanation on Net_Practice and networking basics.

First of all, thanks to [Robin | radelwar](#) for explaining the more in depth stuff to me.

This may not be perfect since I just learned about all of the following myself. So please, if there is anything wrong or missing, please notify me by reaching out to me ([my profile](#)) or create an [issue](#), thanks.

I only created this tutorial, since the project has a few flaws and you have the ability to solve it in a way that's wrong, but the project is still telling you that everything is correct.

And all other tutorials I found used these flaws to simplify the solution.

Contents

- [Basics](#)
 - [special IP-ranges](#)
 - [Masks](#)
 - [Switches](#)
 - [Routers](#)

- [Routing Tables](#)
- [Network](#)
- [Levels](#)
 - [Level 1](#)
 - [Level 2](#)
 - [Level 3](#)
 - [Level 4](#)
 - [Level 5](#)
 - [Level 6](#)
 - [Level 7](#)
 - [Level 8](#)
 - [Level 9](#)
 - [Level 10](#)

🔗 Basics

For this project we only use IPv4, so I won't talk about IPv6.

An IPv4-adress is a 32-bit number divided into 4 "blocks", each 8 bits.

i.e.:

192.168.100.1 turns into 11000000.10101000.01100100.00000001

So the min. value of one "block" is 0 and the max. value is 255 .

The same logic applies to the network-mask:

255.255.255.0 turns into 11111111.11111111.11111111.00000000

Special to the mask is, after one bit was 0 there can't be any 1 bit's anymore.

So the only available numbers are:

- 255 (binary: 11111111)
- 254 (binary: 11111110)
- 252 (binary: 11111100)
- 248 (binary: 11111000)
- 240 (binary: 11110000)
- 224 (binary: 11100000)
- 192 (binary: 11000000)

- 128 (binary: 10000000)
- 0 (binary: 00000000)

Through which 255.255.255.0 is a valid mask
and 255.255.128.128 is **not** a valid mask.

In order to have the ability to send packages between two IP-addresses they either need to be part of the same network or they need to be connected by a router which is part of both subnets.

[back to contents](#)

🔗 Special IP-ranges

The following special address-ranges are reserved for Private Networks:

10.0.0.0 – 10.255.255.255
172.16.0.0 – 172.31.255.255
192.168.0.0 – 192.168.255.255

The following address-range is reserved for so called loopback addresses:

127.0.0.0 – 127.255.255.255

There is some more special ip-ranges, but for this project, you only need to remember those above.

[back to contents](#)

🔗 Masks

The network-mask, subnet-mask or in our project only called mask is there to decide which range of ip-adresses are part of the same subnet.

There are 2 different ways of writing the mask:

- "Dot-decimal notation": 255.255.255.0
- "Class Inter-Domain Routing" or "CIDR": /24

The more usable ip-addresses you need in one subnet, the less subnets you will be able to create.

To help you understanding it, I found this table very helpful:

CIDR	Dot-decimal	Number of IP-addresses per subnet	Usable IP-addresses per subnet	Number of subnets
/32	255.255.255.255	1	0	256
/31	255.255.255.254	2	0	128
/30	255.255.255.252	4	2	64
/29	255.255.255.248	8	6	32
/28	255.255.255.240	16	14	16
/27	255.255.255.224	32	30	8
/26	255.255.255.192	64	62	4
/25	255.255.255.128	128	126	2
/24	255.255.255.0	256	254	1

The number of usable IP-addresses per subnet is lower than the total number of IP's because the first address is reserved as the network-address of the subnet and the last address is reserved as a broadcast-address.

i.e. for mask 255.255.255.252 :

network: 190.3.2.252

broadcast: 190.3.2.255

usable IP's: 190.3.2.253 , 190.3.2.254

[back to contents](#)

🔗 Switches

A switch will enable you to connect more than two devices to the same network.

Its only purpose is to distribute packages to its network.

To see a working example, you can take a look at [Level 3](#).

🔗 Routers

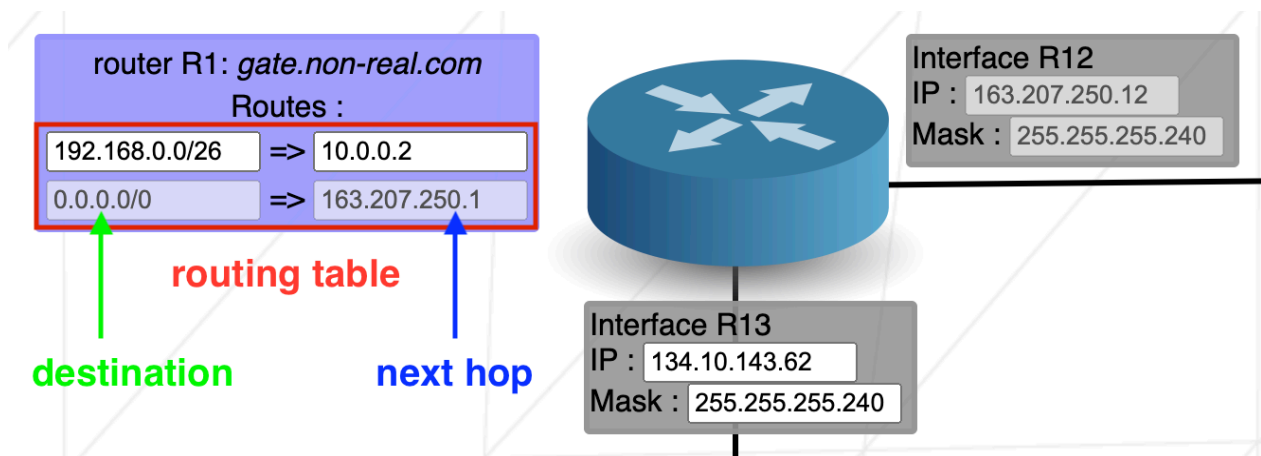
As previously mentioned a router is an interface which enables communication between different networks.

A router has the ability to be part of multiple networks, in Netpractice this is visualized by the so called `Interface` .

If routers and switches are still magic to you, I suggest looking deeper [into it](#) yourself, as their basic understanding is crucial to succeed in this project.

[back to contents](#)

🔗 Routing Table



The routing table is there to store all the different paths to all the networks, the device is part of.

In Net_Practice the routing table consists of two elements, the **destination** and the **next hop**

The **destination** consists of the network-address that you want to send a package to, combined with the CIDR of that network: `190.3.2.252/30` . If you don't want to specify a destination, you can just set it to `default` or `0.0.0.0/0` .

The **next hop** is the address of the next router that you need to send the packages to in order to reach the destination-network.

[back to contents](#)

🔗 Network

And now to connect all of the above mentioned topics.

In order to have a functioning network, you now need to apply all of the parts talked about earlier.

If there should be a working connection in a network, the devices somehow need to be connected, either directly or with the help of routers which are part of both networks.

Now you may ask, how do I know if two devices are part of the same network? For this you need to combine the IP-address and the mask of the devices in order to get the network-address, that device is part of.

By combining I mean, doing a bit-by-bit-AND-operation.

For that we first need to translate the IP and the mask to binary.

i.e.:

IP: 192.168.100.1 in binary: 11000000.10101000.1100100.00000001

MASK: 255.255.255.0 in binary: 11111111.11111111.11111111.00000000

Now you just combine the two bit by bit, if both bits are a 1 the corresponding bit of the network-address is 1, in any other case the corresponding bit is 0.

By doing that to the mentioned example, you should get the network-address of

11000000.10101000.1100100.00000000 in binary or 192.168.100.0 in dot-decimal.

If two devices share the same network-address, they are part of the same network and communication is ensured.

[back to contents](#)

Levels

Here are all the solutions and explanations for all 10 Levels.

Level 1

► show

[↗](#) **Level 2**

▶ show

[↗](#) **Level 3**

▶ show

[↗](#) **Level 4**

▶ show

[↗](#) **Level 5**

▶ show

[↗](#) **Level 6**

▶ show

[↗](#) **Level 7**

▶ show

[↗](#) **Level 8**

▶ show

[↗](#) **Level 9**

▶ show

Level 10

► show

[back to contents](#)